Module 3

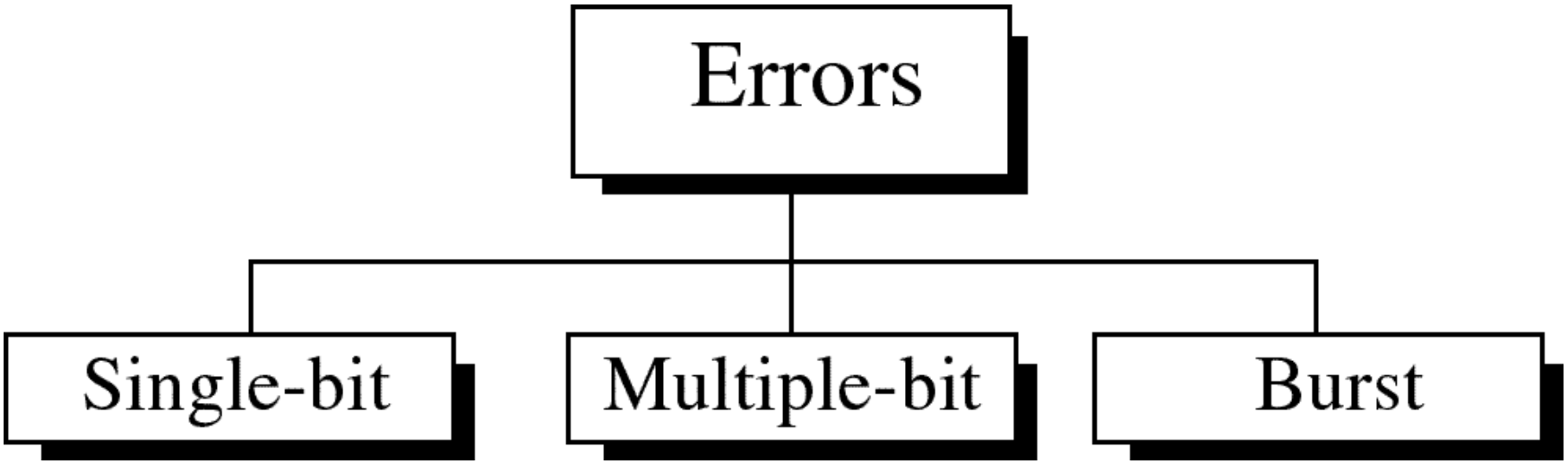
Error Detection and Correction

- **Types of Errors**
- **Detection**
- **Correction**

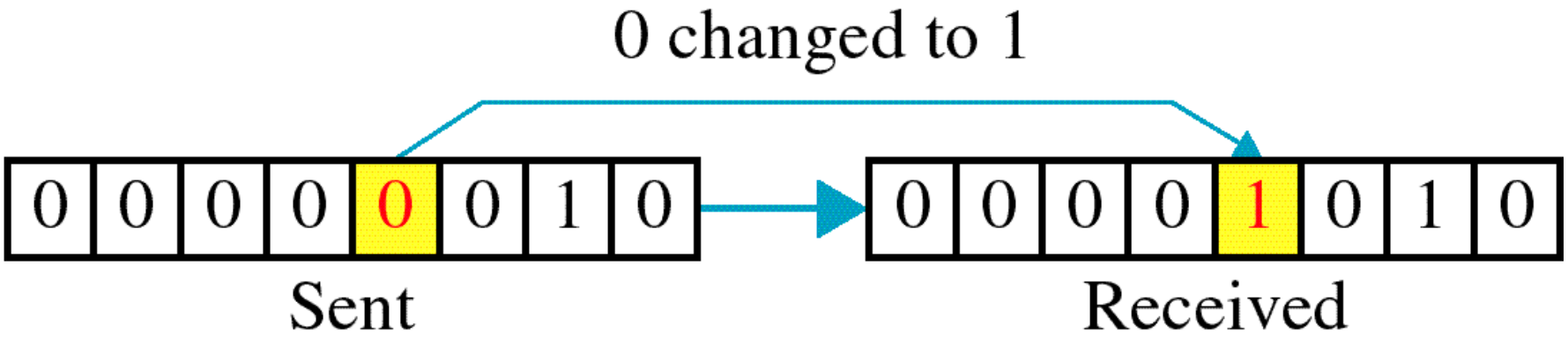
Basic concepts

- ★ Networks must be able to transfer data from one device to another with complete accuracy.
- ★ Data can be corrupted during transmission.
- ★ For reliable communication, errors must be detected and corrected.
- ★ **Error detection and correction** are implemented either at the **data link layer** or the **transport layer** of the OSI model.

Types of Errors

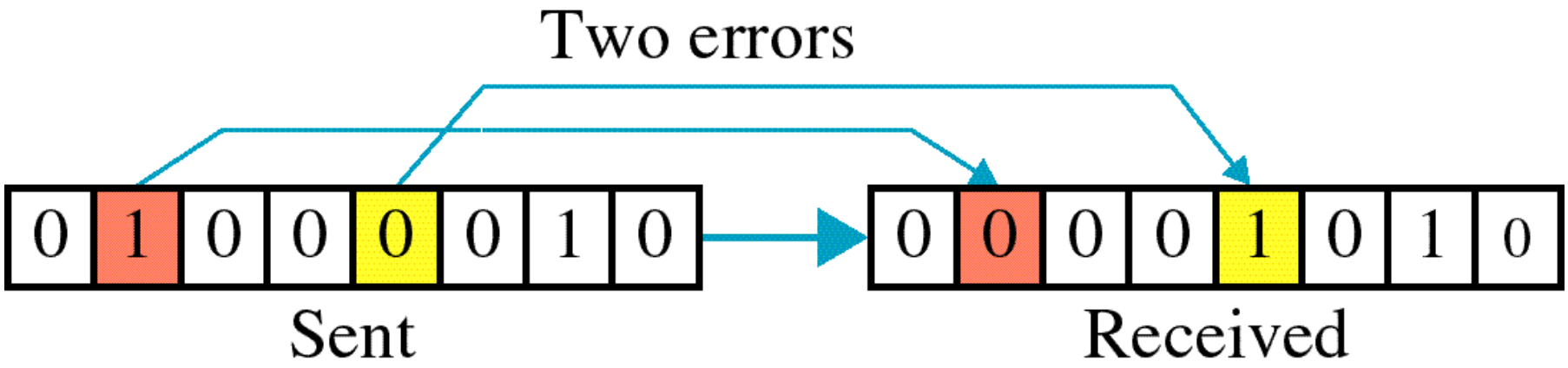


Single-bit error



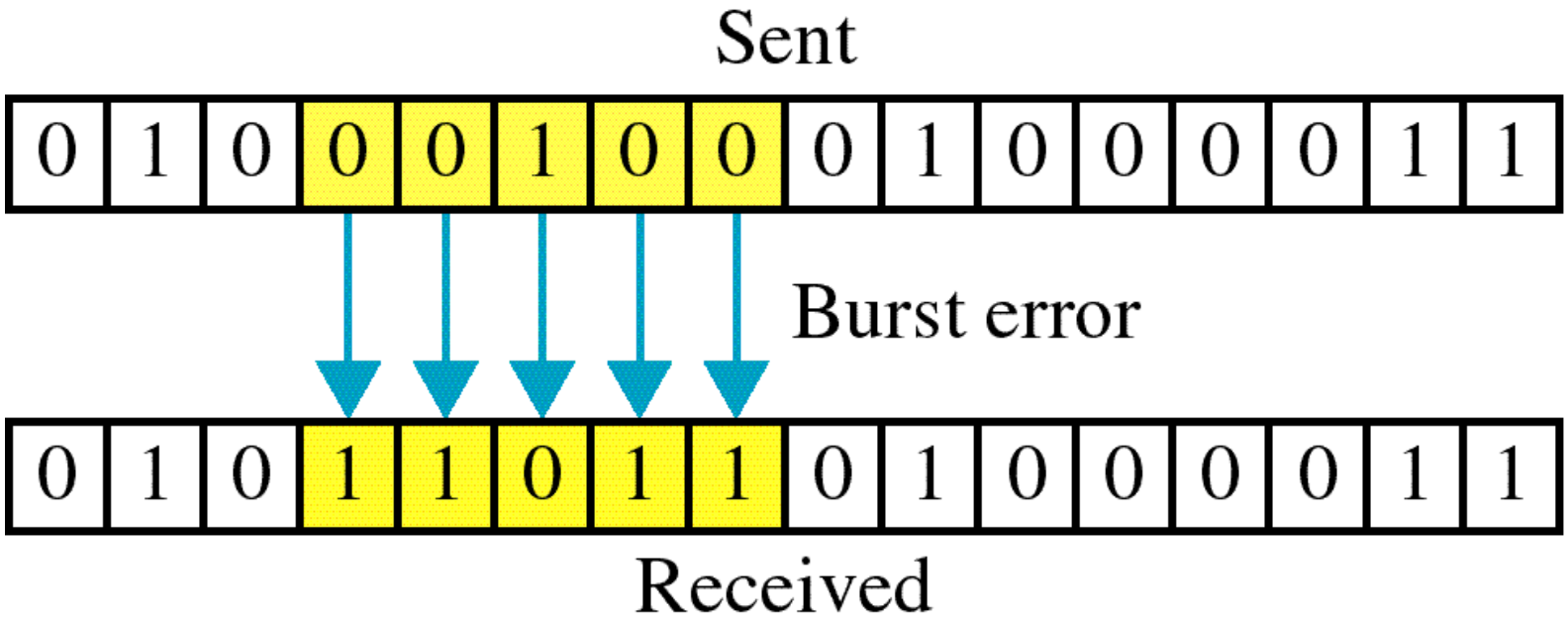
Single bit errors are the **least likely** type of errors in serial data transmission because the noise must have a very short duration which is very rare. However this kind of errors occur in **parallel transmission.**

Multiple-bit error



More than **one bit received** in the frame is found to be corrupted. Refer to the below image for the **multiple-bit error**

Burst error


Sent
0 1 0 0 0 1 0 0 0 1 0 0 0 0 1 1
Burst error
0 1 0 1 1 0 1 1 0 1 0 0 0 0 1 1
Received

The term **burst error** means that two or more bits in the data unit have changed from 1 to 0 or from 0 to 1.

More than one consecutive bit is corrupted in the received frame.

The length of the burst is measured from the first corrupted bit to the last corrupted bit.

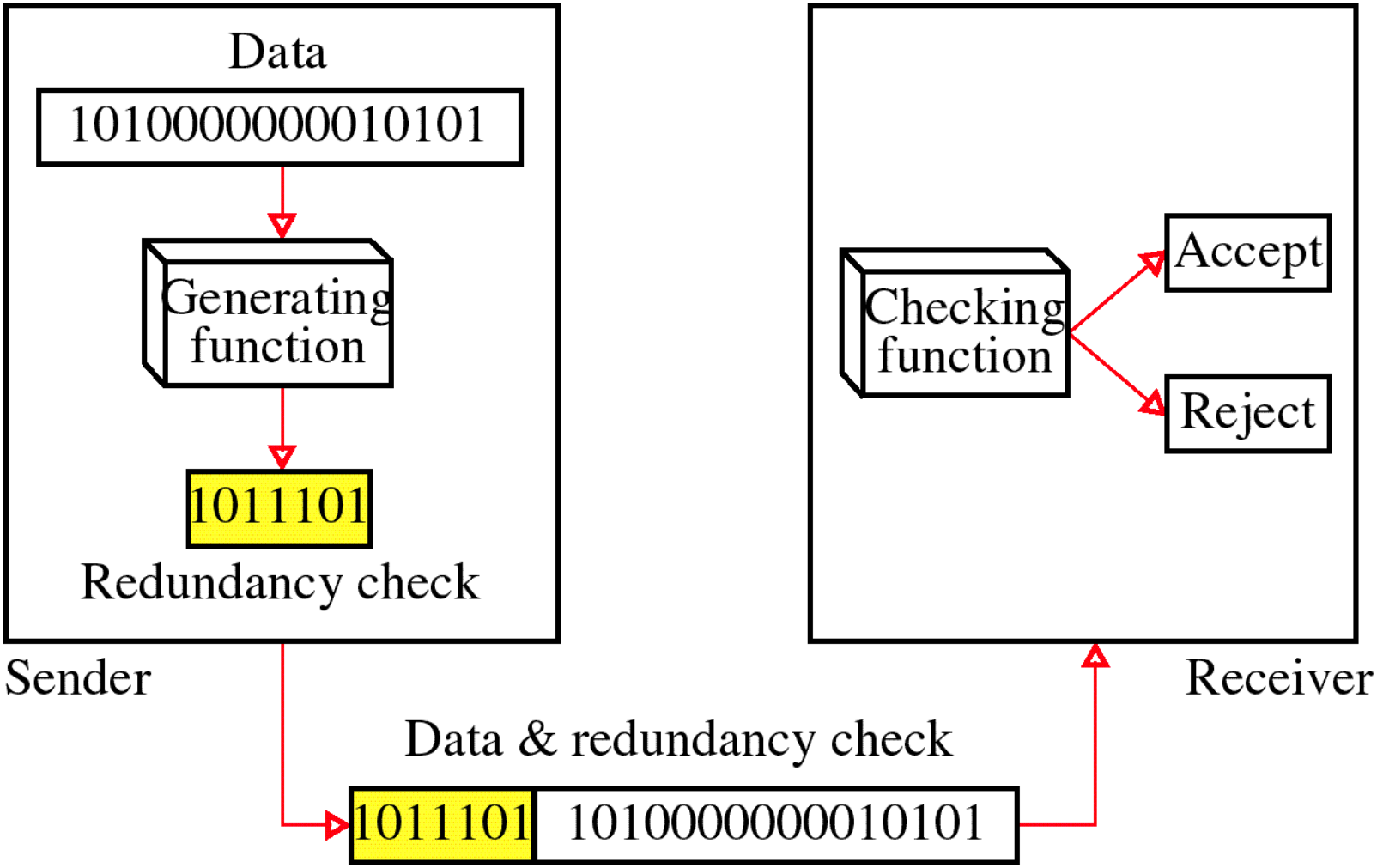
- **Burst error is most likely to happen in serial transmission** since the duration of noise is normally longer than the duration of a bit.
- The number of bits affected depends on the data rate and duration of noise.

Error detection

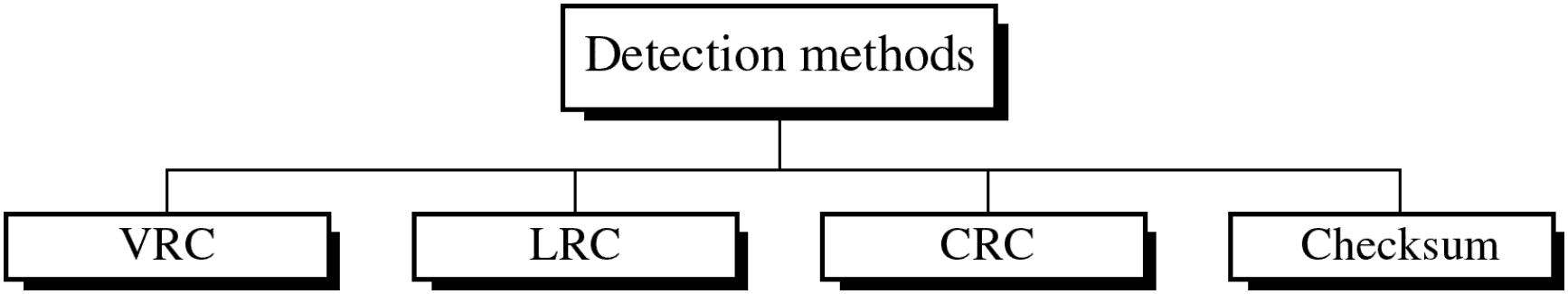
Error detection means to decide whether the received data is correct or not without having a copy of the original message.

Error detection **uses the concept of redundancy**, **which means** adding extra bits for detecting errors at the destination.

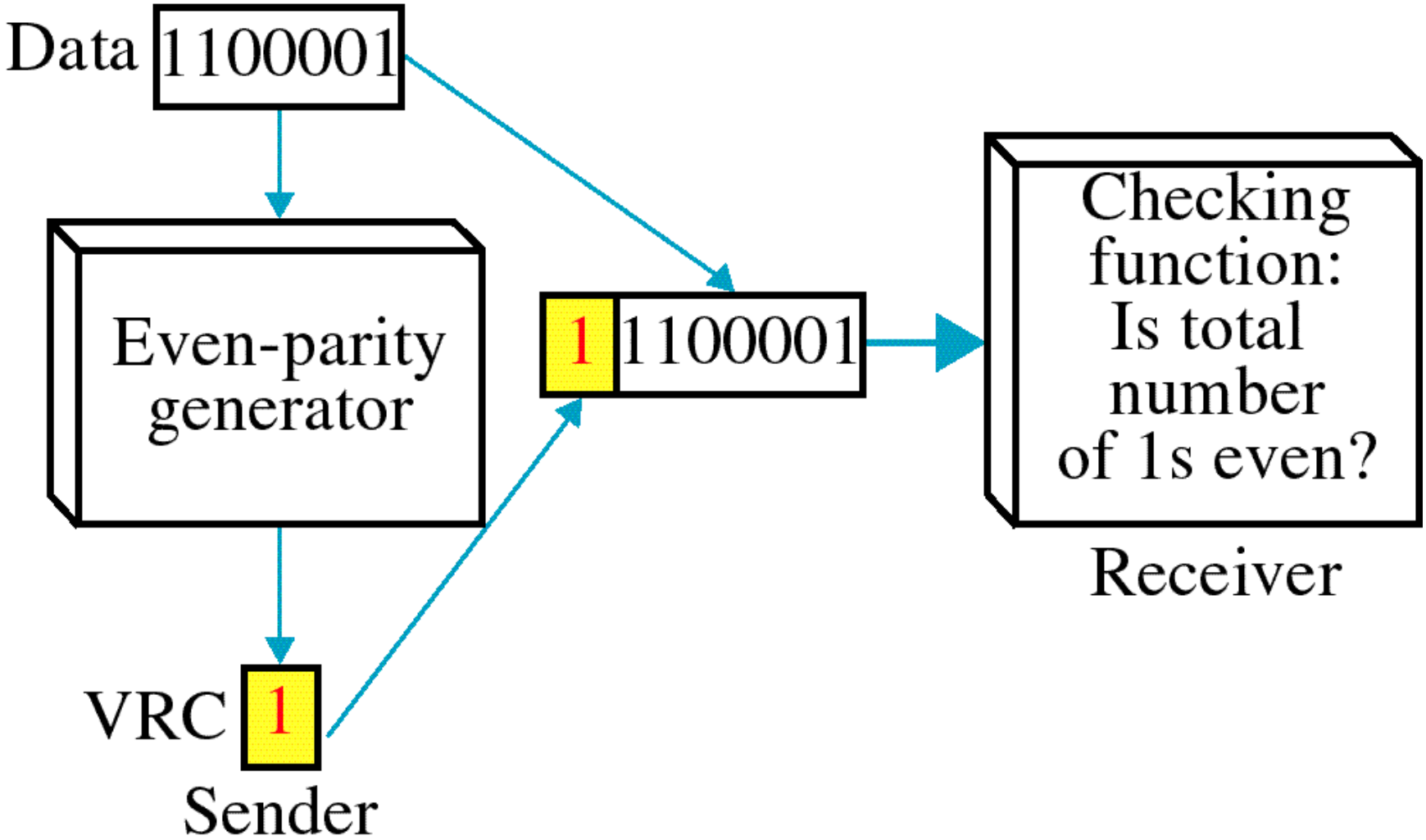
Redundancy


Data
1010000000010101
Generating function
1011101
Redundancy check
Sender
Checking function
Accept
Reject
Receiver
Data & redundancy check
1011101
1010000000010101

Four types of redundancy checks are used in data communications

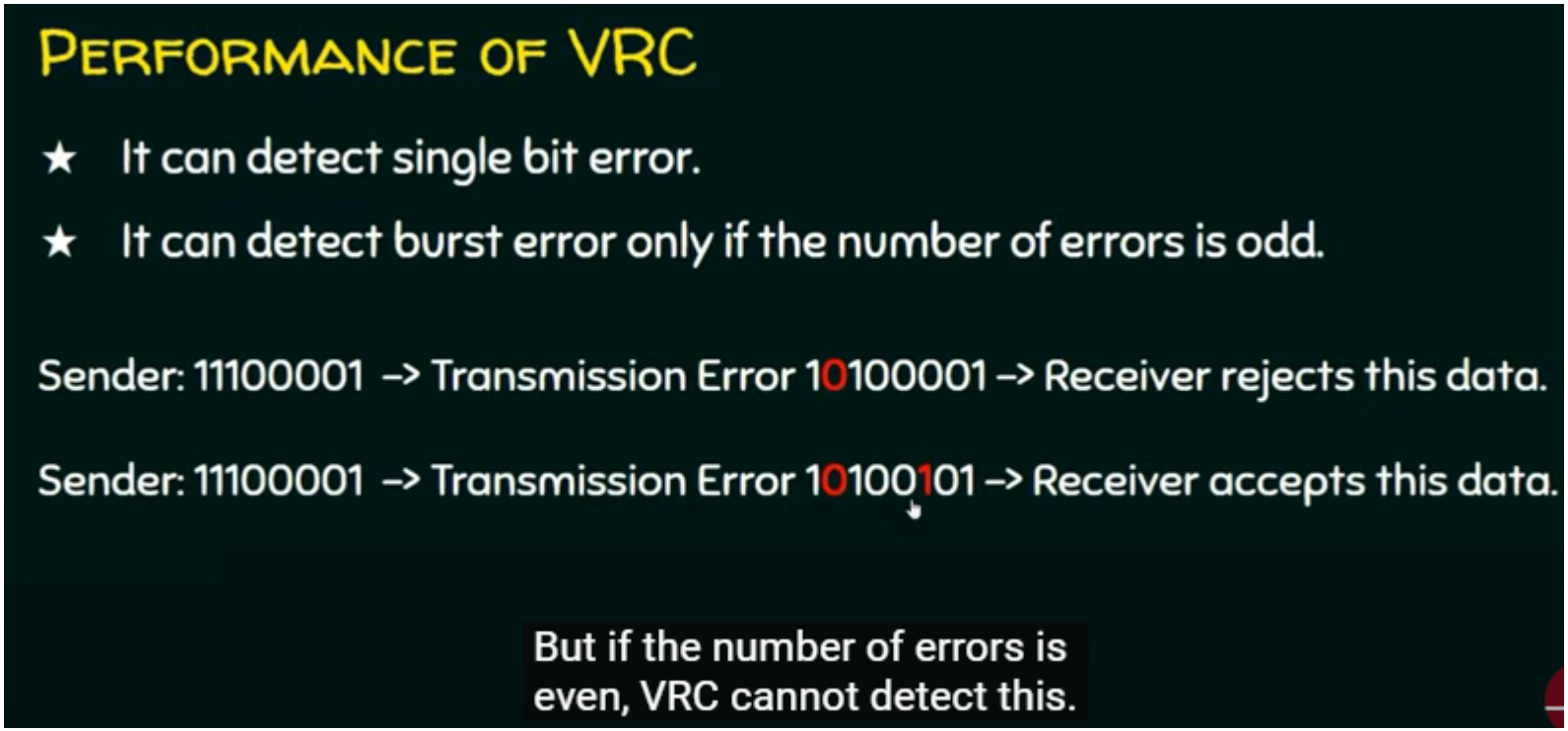


Vertical Redundancy Check
VRC



Performance

- It can detect single bit error
- It can detect burst errors only if the total number of errors is odd.



Homework

Append the parity bit after each block shown below using VRC.

1110110 1101111 1110010

LRC

LRC

- ★ Longitudinal Redundancy Check.
- ★ In LRC, a block of bits is organized in rows and columns.
- ★ a.k.a Two Dimensional parity.
- ★ The parity bit is calculated for each column and sent along with the data.
- ★ The block of parity acts as the redundant bits.

LRC – Example

Find the LRC for the data blocks 11100111 11011101 00111001 10101001 and determine the data that is transmitted?

Odd no. of 1's	1
Even no. of 1's	0

1	1	1	0	0	1	1	1
1	1	0	1	1	1	0	1
0	0	1	1	1	0	0	1
1	0	1	0	1	0	0	1
1	0	1	0	1	0	1	0

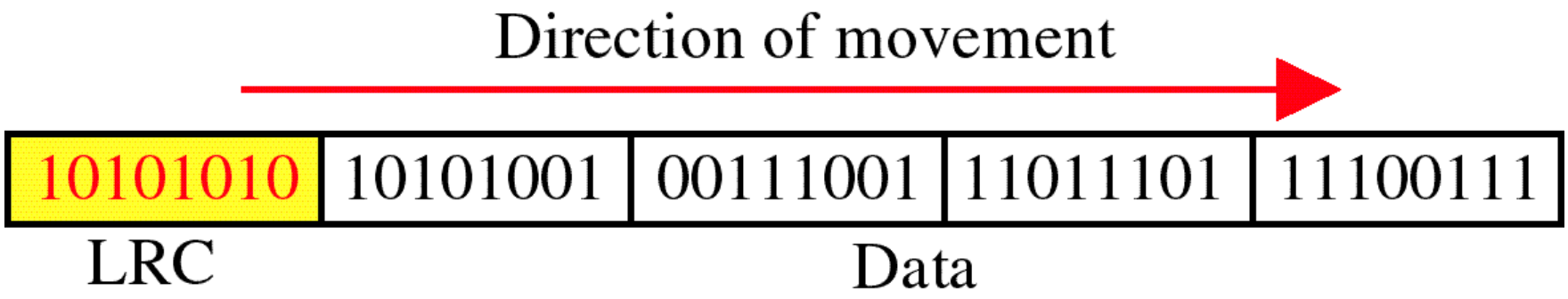
LRC →

Direction of movement →

10101010	10101001	00111001	11011101	11100111

LRC Data

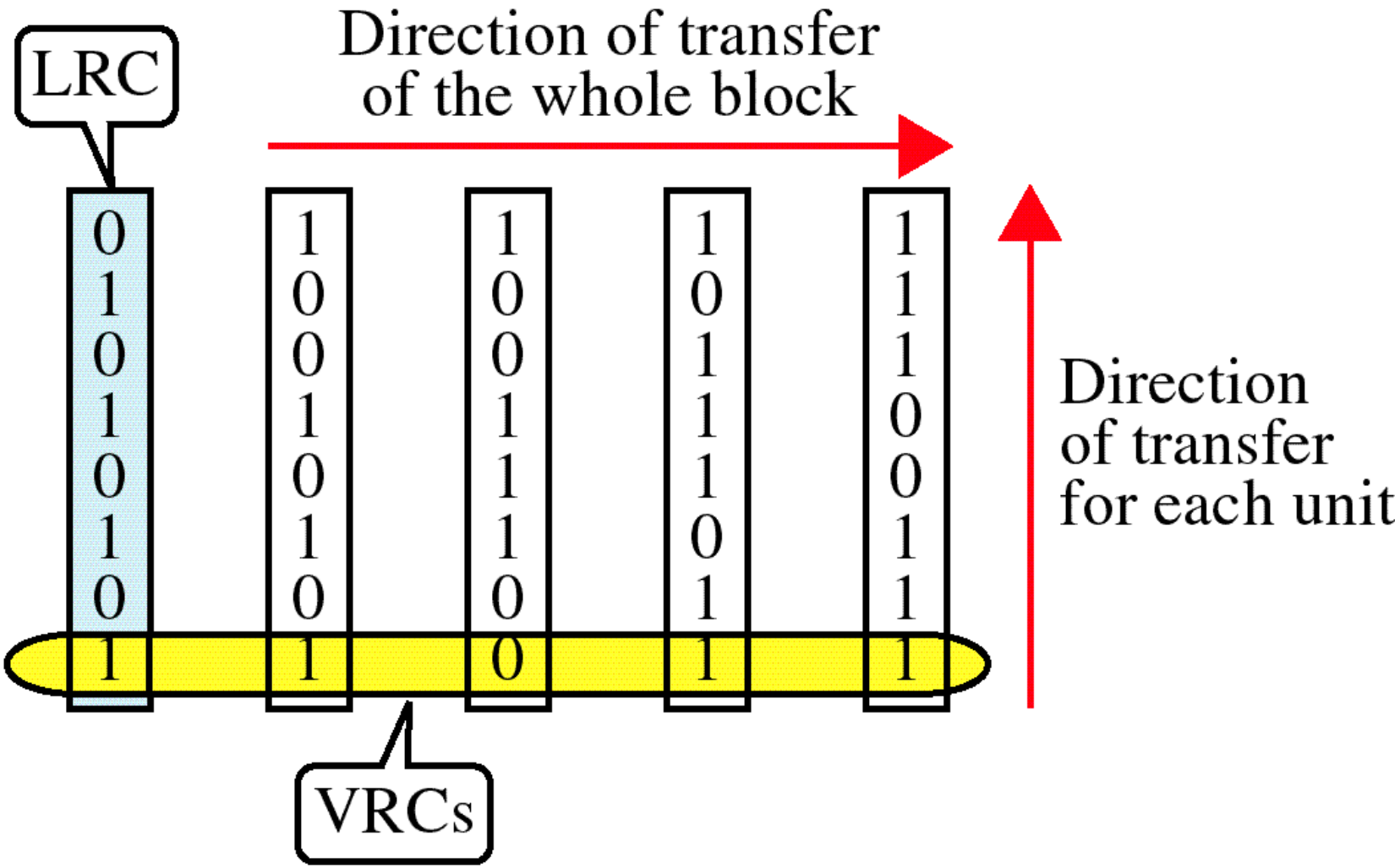
Longitudinal Redundancy Check
LRC



Performance

- LCR increases the likelihood of detecting burst errors.
- If two bits in one data units are damaged and two bits in exactly the same positions in another data unit are also damaged, the LRC checker will not detect an error.

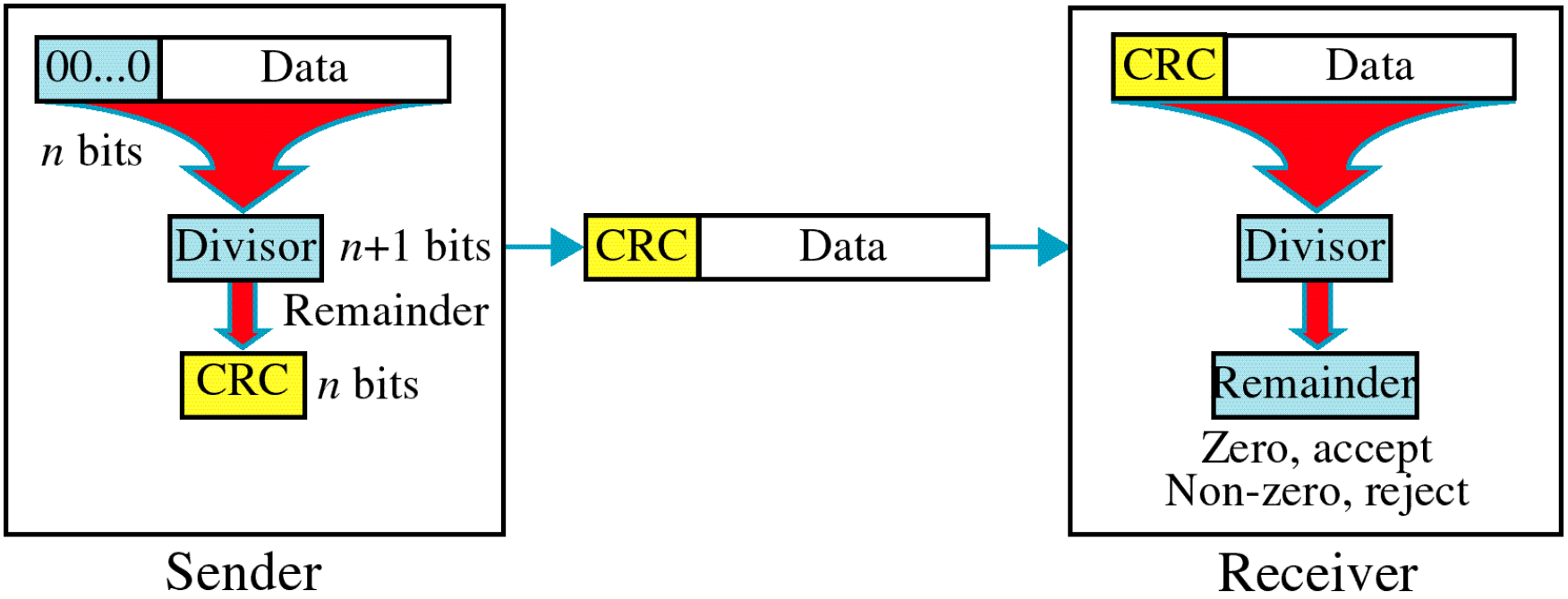
VRC and LRC


LRC
Direction of transfer
of the whole block
0 1 0 1 0 1 0 1
1 0 0 1 0 1 0 1
1 0 0 1 1 1 0 0
1 0 1 1 1 0 1 1
1 1 1 0 0 1 1 1
Direction
of transfer
for each unit
VRCs

Homework

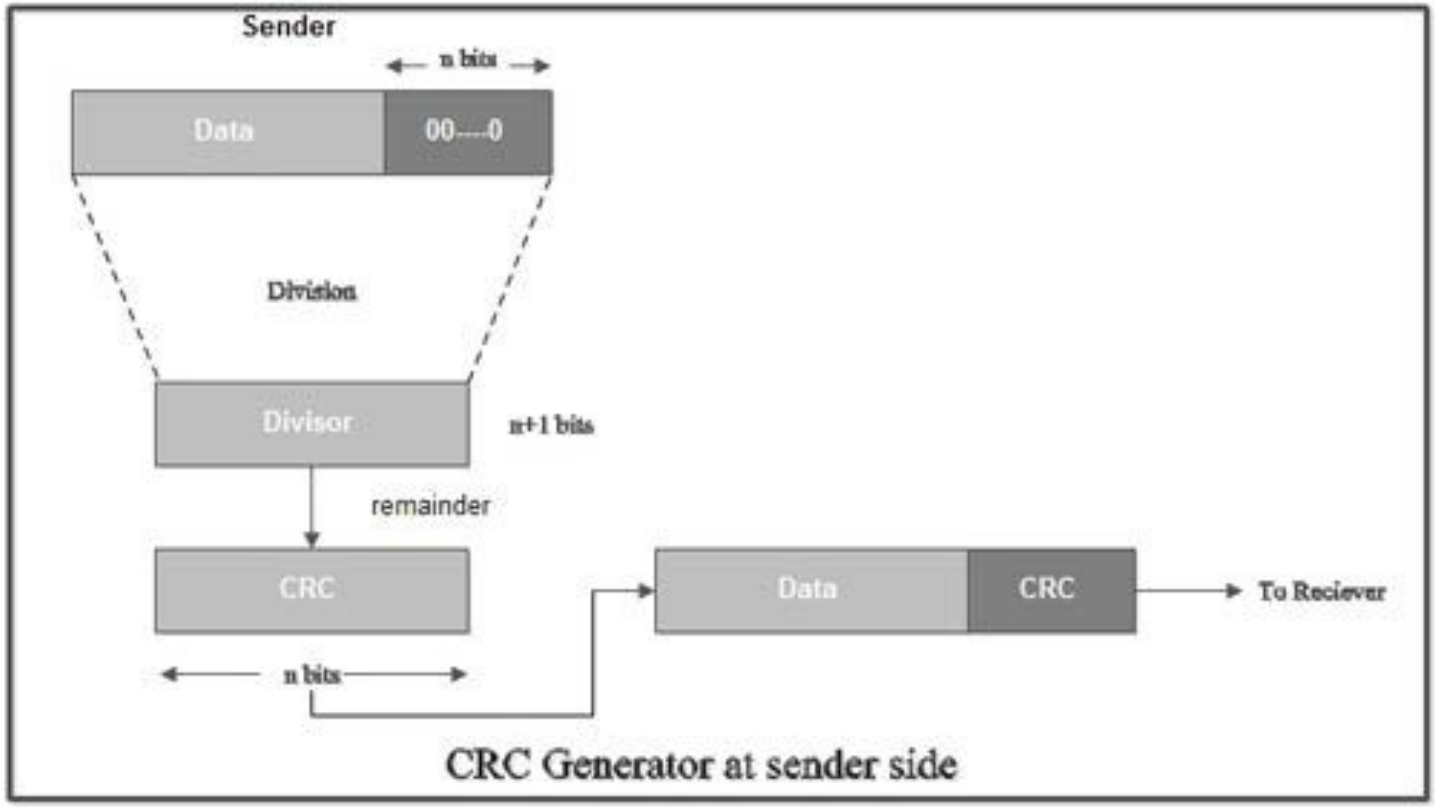
Find the LRC for the data blocks 01110111 10101001 01101001 10101010 and determine the data that is transmitted.

Cyclic Redundancy Check
CRC

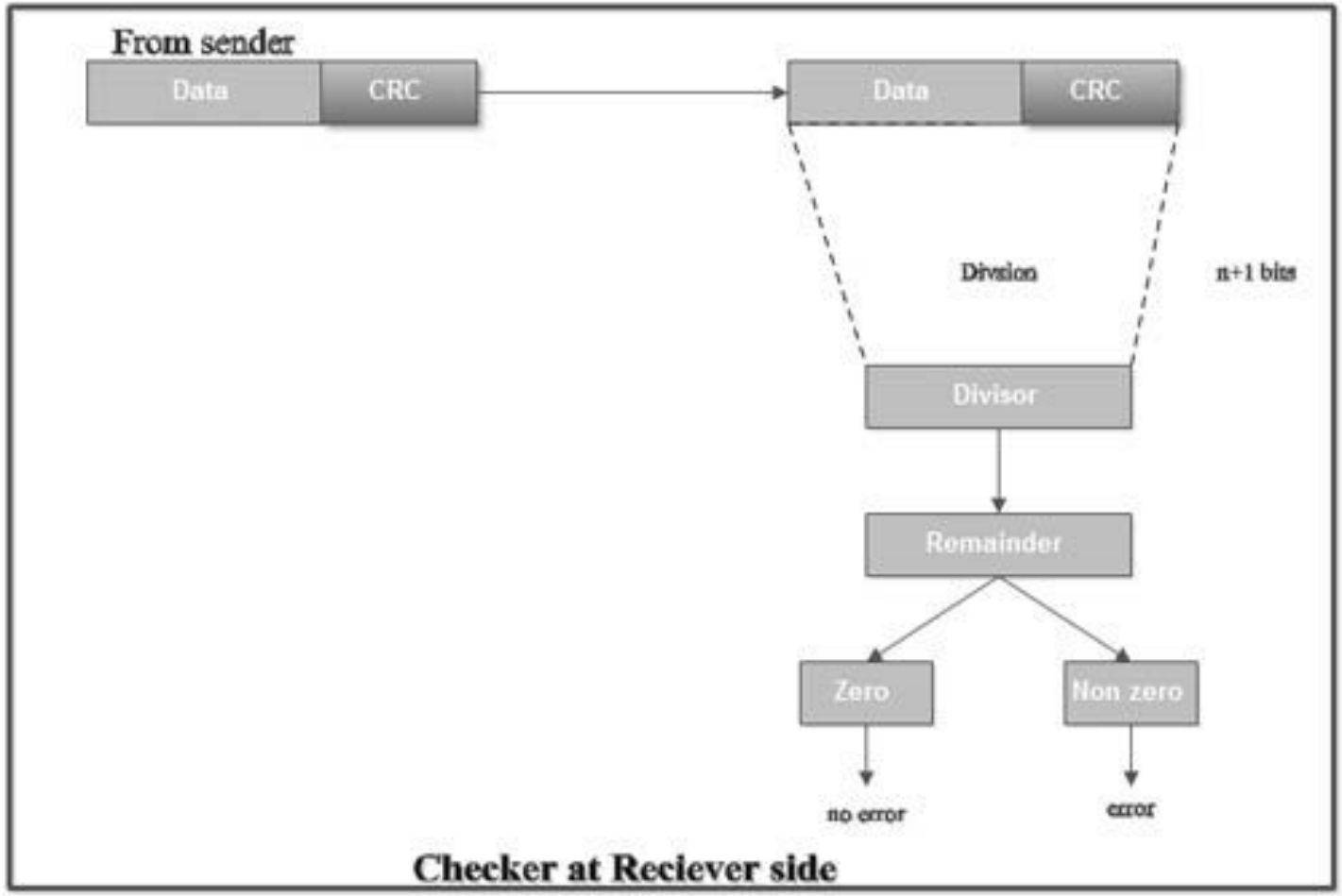


Cyclic Redundancy Check

- Given a k-bit frame or message, the transmitter generates an n-bit sequence, known as a frame check sequence (FCS), so that the resulting frame, consisting of (k+n) bits, is exactly divisible by some predetermined number.
- The receiver then divides the incoming frame by the same number and, if there is no remainder, assumes that there was no error.



CRC Generator at sender side


From sender
Data
CRC
Data
CRC
Division
n+1 bits
Divisor
Remainder
Zero
Non zero
no error
error
Checker at Reciever side

1. Data unit 1011000 is divided by 1011.

```
                    1 0 1 0  (Quotient)
Divisor :      1011 ) 1 0 0 1 [0 0 0] <-- appended zeroes : 3 bits
4 bits                1 0 1 1
                      -------
                        0 1 0 0
use divisor 0000 ->     0 0 0 0
when leftmost bit is 0  -------
                          1 0 0 0
                          1 0 1 1
                          -------
                            0 1 1 0
use divisor 0000 ->         0 0 0 0
when leftmost bit is 0      -------
                            [1 1 0] <-- Remainder : 3 bits
```

Data	CRC (3 bits)
1001	110

CRC generated (Binary division)

. The remainder obtained is 000; it means there is no error.

```
                1 0 1 0
1011 ) 1 0 0 1 1 1 0
       1 0 1 1
       -------
         0 1 0 1
Left most bit : 0 ->
divisor : 0000 ->  0 0 0 0
                   -------
                     1 0 1 1
                     1 0 1 1
                     -------
                       0 0 0 0
                       0 0 0 0
                       -------
                       [0 0 0]  Remainder is zero, no error
```

CRC decoded (binary division)

Data word to be sent - 100100

Key - 1101 [Or generator polynomial $x^3 + x^2 + 1$]

Sender Side:

```
          111101
1101 | 100100000 |
       1101
       ----
        1000
        1101
        ----
         1010
         1101
         ----
          1110
          1101
          ----
           0110
           0000
           ----
            1100
            1101
            ----
             001
             ---
```

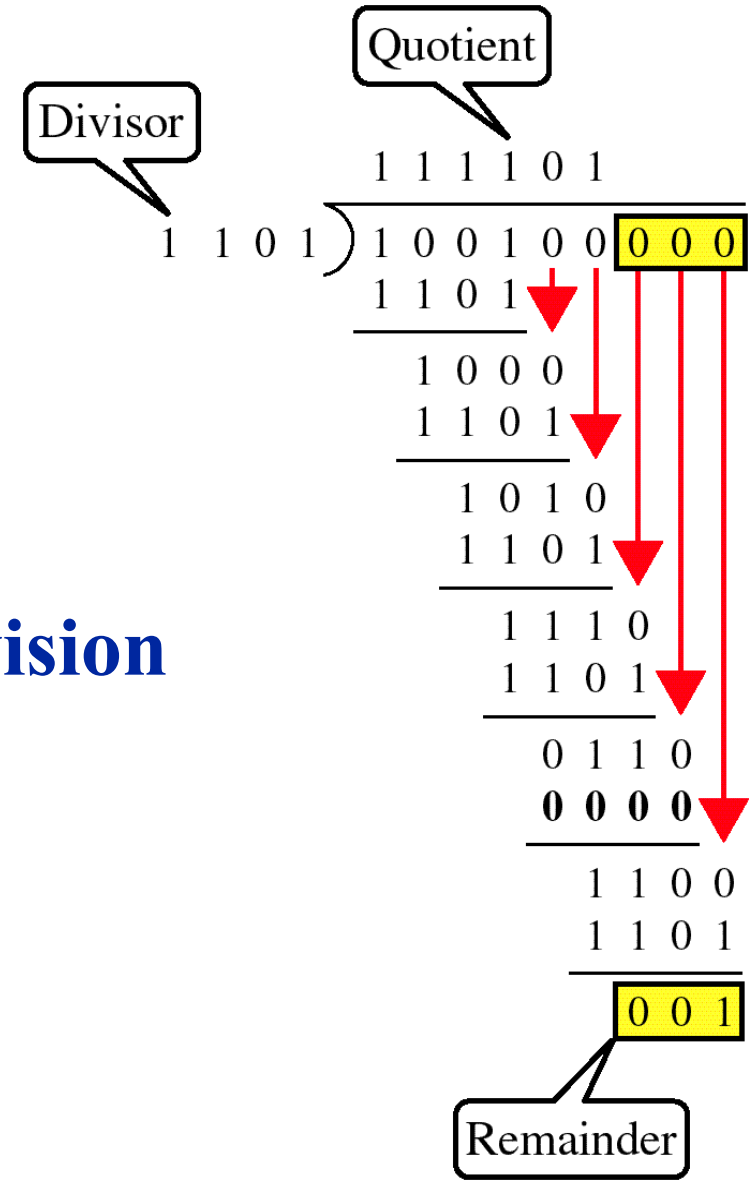
Receiver Side:

Code word received at the receiver side 100100001

```
        111101
1101 | 100100001 |
       1101
       ----
        1000
        1101
        ----
         1010
         1101
         ----
          1110
          1101
          ----
          0110
          0000
          ----
           1101
           1101
           ----
           0000
           ----
```

Therefore, the remainder is all zeros. Hence, the data received has no error.

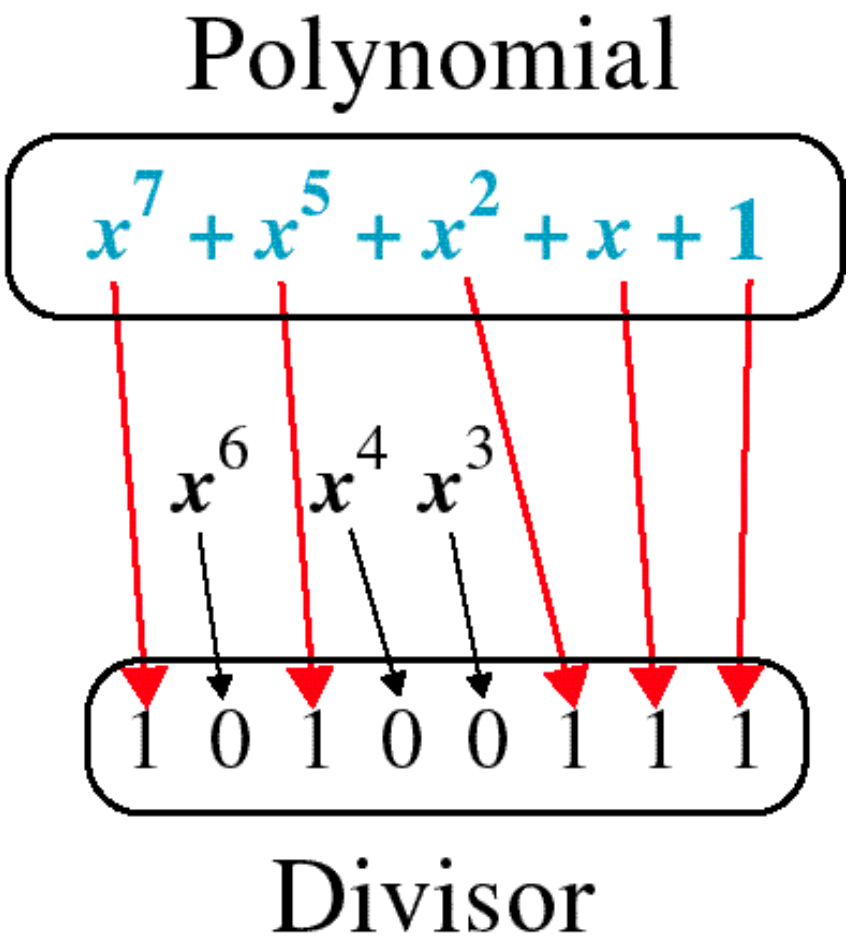
Binary Division

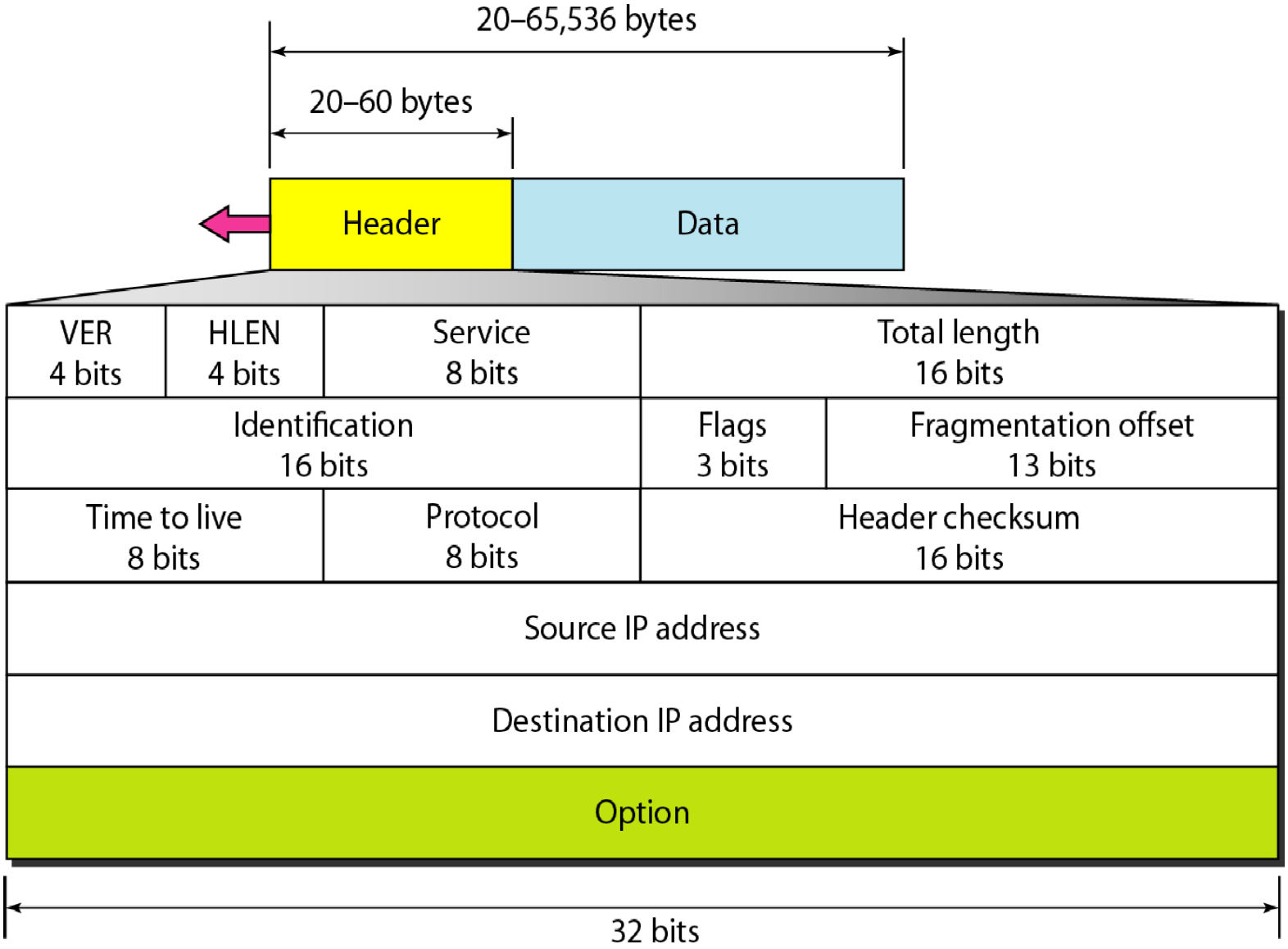


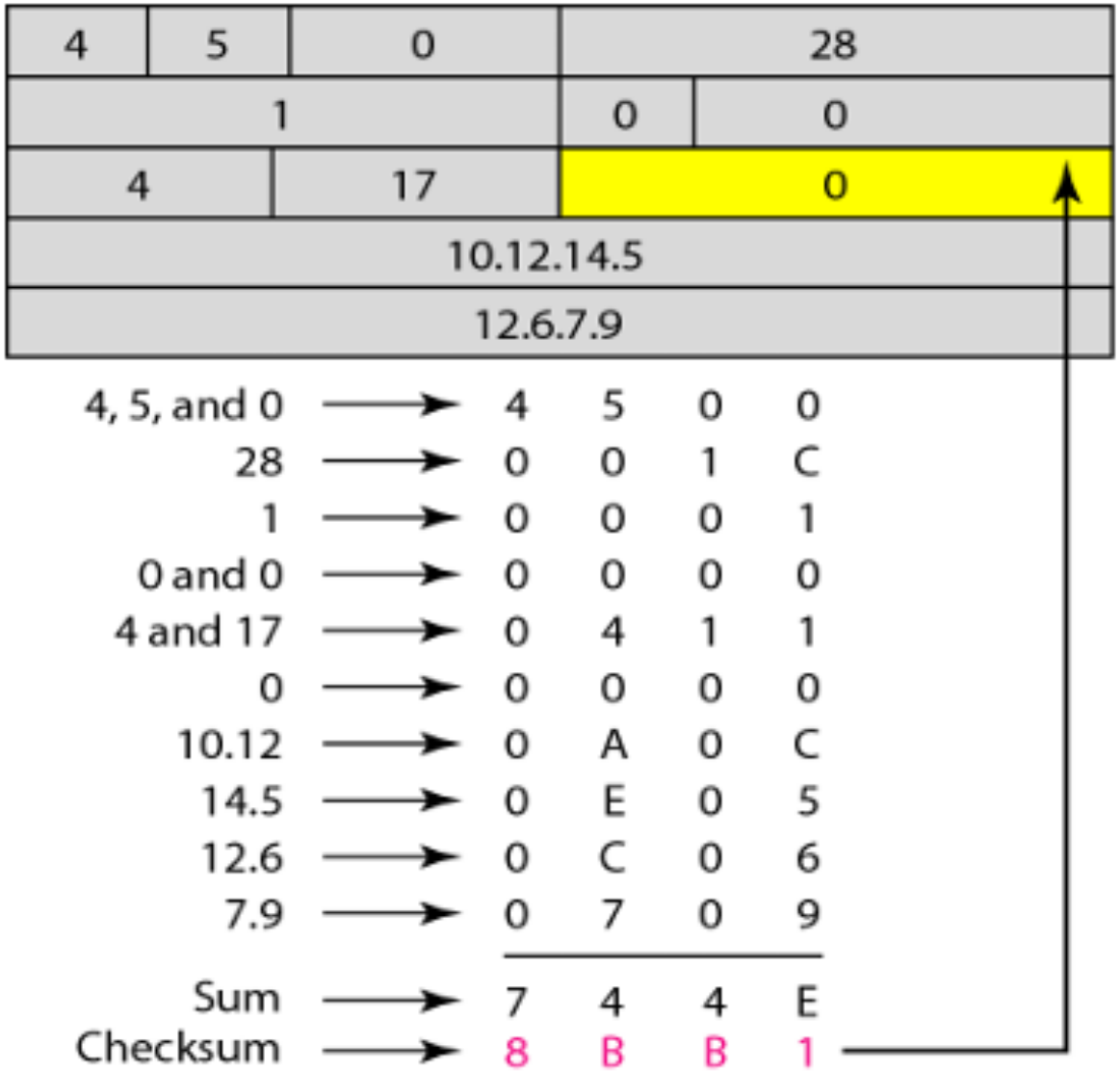
Polynomial

$$x^7 + x^5 + x^2 + x + 1$$

Polynomial and Divisor



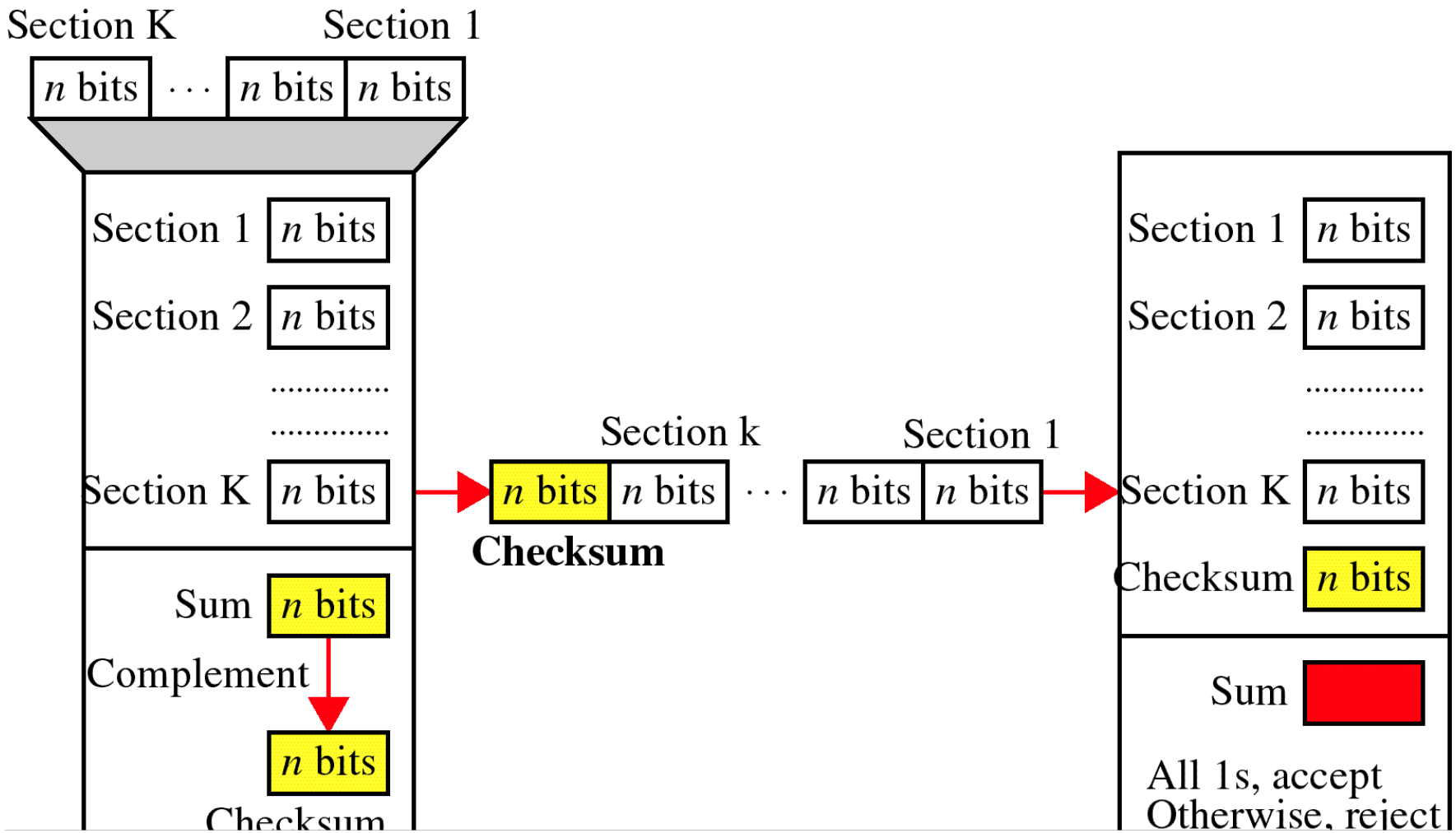

20–65,536 bytes
20–60 bytes
Header
Data
VER
4 bits
HLEN
4 bits
Service
8 bits
Total length
16 bits
Identification
16 bits
Flags
3 bits
Fragmentation offset
13 bits
Time to live
8 bits
Protocol
8 bits
Header checksum
16 bits
Source IP address
Destination IP address
Option
32 bits


4
5
0
28
1
0
0
4
17
0
10.12.14.5
12.6.7.9
4, 5, and 0 → 4 5 0 0
28 → 0 0 1 C
1 → 0 0 0 1
0 and 0 → 0 0 0 0
4 and 17 → 0 4 1 1
0 → 0 0 0 0
10.12 → 0 A 0 C
14.5 → 0 E 0 5
12.6 → 0 C 0 6
7.9 → 0 7 0 9
Sum → 7 4 4 E
Checksum → 8 B B 1

Checksum

- **Checksum** is a error detection which detects the error by dividing the data into the segments of equal size and then **use 1's complement** to find the sum of the segments and then sum is transmitted with the data to the receiver and same process is done by the receiver and at the receiver side, all **zeros** in the sum indicates the correctness of the data.
- First of all data is divided into **k segments** in a checksum error detection scheme and each segment has m bits.
- For finding out the sum at the sender's side, all segments are added through **1's complement arithmetic**. And for determining the checksum we complement the sum.
- Along with data segments, the checksum segments are also transferred.
- All the segments that are received on the receiver's side are added through 1S complement arithmetic to determine the sum. Then complement the sum also.
- The received data is accepted only on the condition that the result is found to be **0**. And if the result is not 0 then it will be discarded.

Checksum


Section K
Section 1
n bits
. . .
n bits
n bits
Section 1
n bits
Section 2
n bits
Section K
n bits
Sum
n bits
Complement
n bits
Checksum
Section k
Section 1
n bits
n bits
. . .
n bits
n bits
Checksum
Section 1
n bits
Section 2
n bits
Section K
n bits
Checksum
n bits
Sum
All 1s, accept
Otherwise, reject

Checksum

Original Data

10011001	11100010	00100100	10000100
1	*2*	*3*	*4*

k=4, m=8

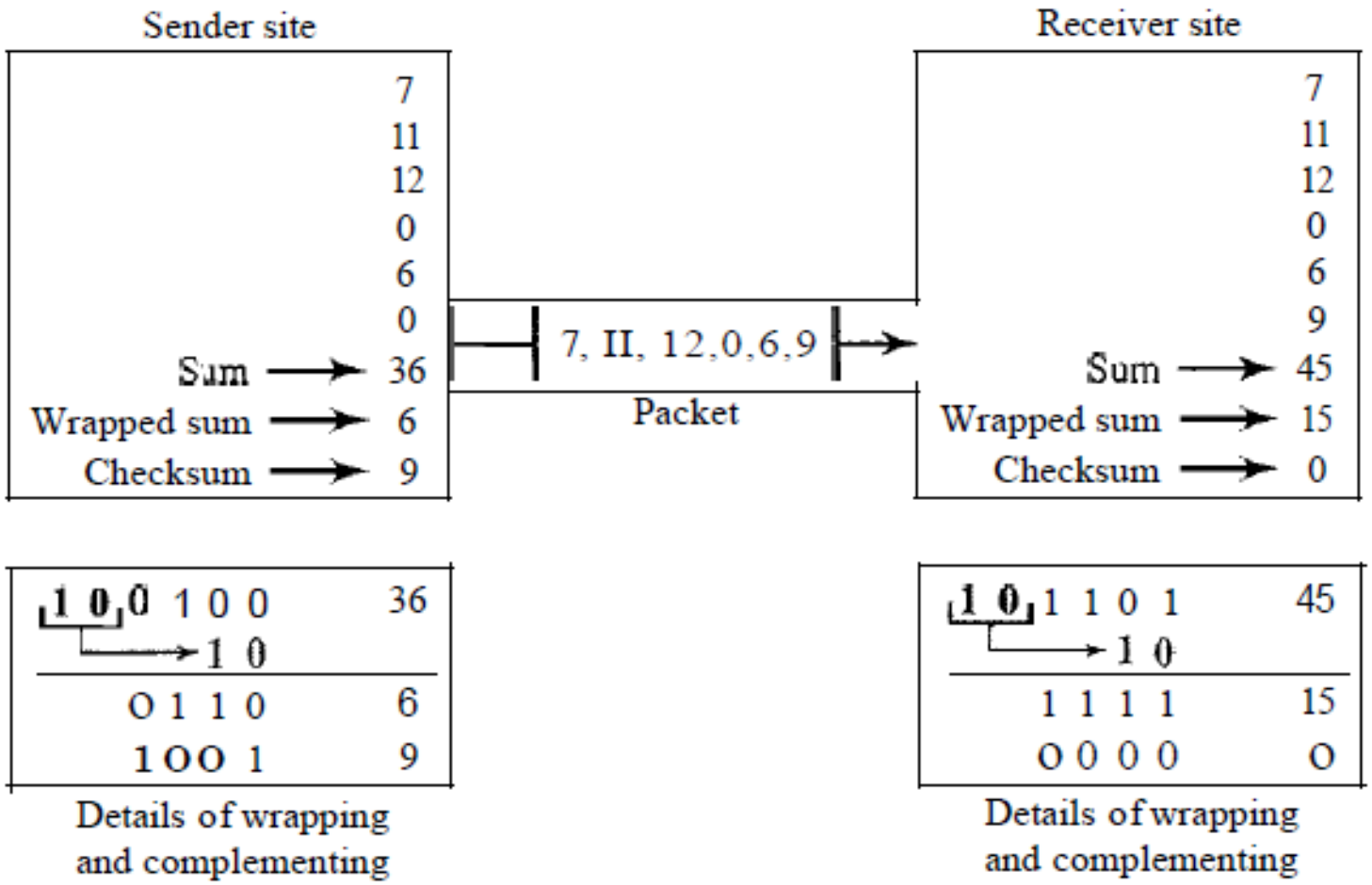
Sender

```
1        10011001
2        11100010
         ---------
         101111011
         ↳       1
         ---------
         01111100
3        00100100
         ---------
         10100000
4        10000100
         ---------
         100100100
         ↳       1
         ---------
Sum:     00100101
         ---------
CheckSum: 11011010
```

Receiver

```
1        10011001
2        11100010
         ---------
         101111011
         ↳       1
         ---------
         01111100
3        00100100
         ---------
         10100000
4        10000100
         ---------
         100100100
         ↳       1
         ---------
         00100101
         11011010
         ---------
Sum:     11111111
Complement: 00000000
```

Conclusion : Accept Data


Sender site
7
11
12
0
6
0
Sum
36
Wrapped sum
6
Checksum
9
7, II, 12,0,6,9
Packet
Receiver site
7
11
12
0
6
9
Sum
45
Wrapped sum
15
Checksum
0
1 0 0 1 0 0 36
1 0
0 1 1 0 6
1 0 0 1 9
Details of wrapping
and complementing
1 0 1 1 0 1 45
1 0
1 1 1 1 15
0 0 0 0 0
Details of wrapping
and complementing

Example 10.23

Let us calculate the checksum for a text of 8 characters ("Forouzan"). The text needs to be divided into 2-byte (16-bit) words. We use ASCII (see Appendix A) to change each byte to a 2-digit hexadecimal number. For example, F is represented as Ox46 and o is represented as Ox6F. Figure 10.25 shows how the checksum is calculated at the sender and receiver sites. In part a of the figure,

1	0	1	3		Carries
	4	6	6	F	(Fo)
	7	2	6	F	(ro)
	7	5	7	A	luz)
	6	1	6	E	(an)
	0	0	0	0	Checksum (initial)
	8	F	C	6	Sum (partial)
				1	
	8	F	C	7	Sum
	7	0	3	8	Checksum (to send)

a. Checksum at the sender site

1	0	1	3		Carries
	4	6	6	F	IFo)
	7	2	6	F	(ro)
	7	5	7	A	(uz)
	6	1	6	E	(an)
	7	0	3	8	Checksum (received)
	F	F	F	E	Sum (partial)
				1	
	F	F	F	F	Sum
	0	0	0	0	Checksum (new}

b. Checksum at the receiver site

At the sender

- The unit is divided into *k* sections, each of *n* bits.
- All sections are added together using one's complement to get the sum.
- The sum is complemented and becomes the checksum.
- The checksum is sent with the data

At the receiver

- The unit is divided into k sections, each of n bits.
- All sections are added together using one's complement to get the sum.
- The sum is complemented.
- If the result is zero, the data are accepted: otherwise, they are rejected.

Performance

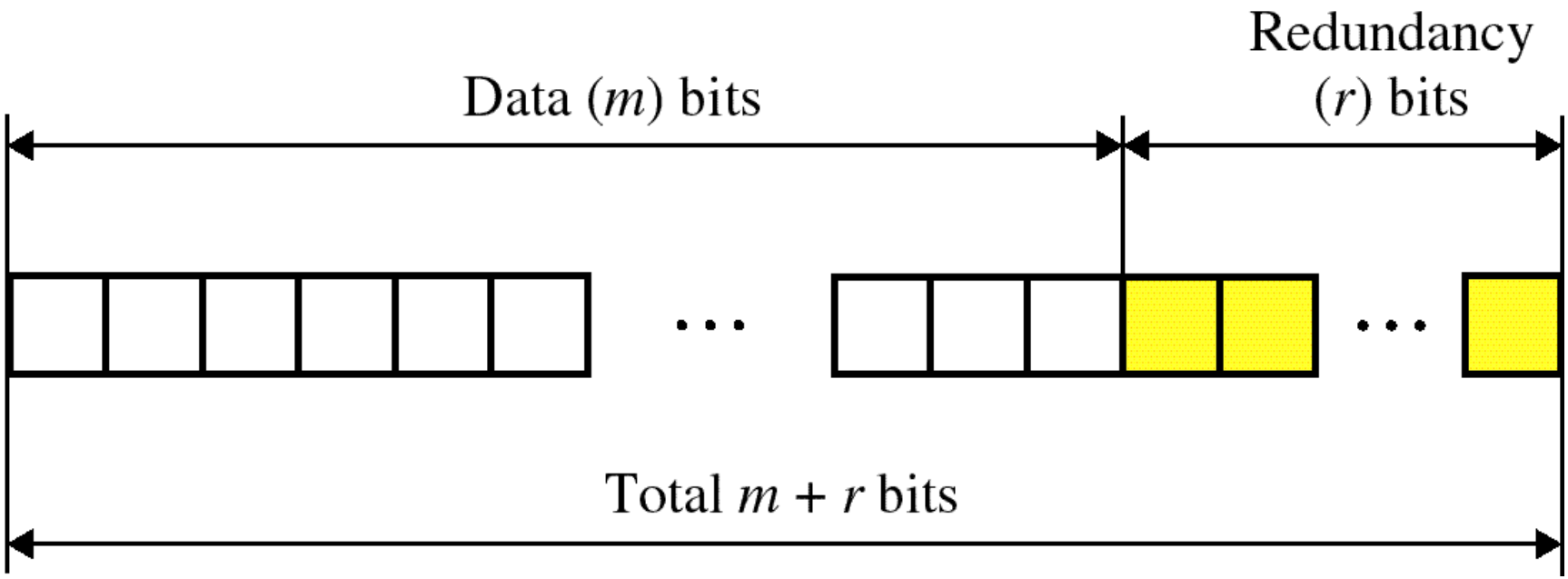
- The checksum detects all errors involving an odd number of bits.
- It detects most errors involving an even number of bits.
- If one or more bits of a segment are damaged and the corresponding bit or bits of opposite value in a second segment are also damaged, the sums of those columns will not change and the receiver will not detect a problem.

Error Correction

It can be handled in two ways:

1) receiver can have the sender's address to retransmit the entire data unit.
2) The receiver can use an error-correcting code, which automatically corrects certain errors.

Error Correction



Hamming Code

- Hamming code is a block code that is capable of detecting up to two simultaneous bit errors and correcting single-bit errors.
- In this coding method, the source encodes the message by inserting redundant bits within the message.
- These redundant bits are extra bits that are generated and inserted at specific positions in the message itself to enable error detection and correction.
- When the destination receives this message, it performs recalculations to detect errors and <u>find the bit position that has error.</u>

Encoding a message by Hamming Code

- The procedure used by the sender to encode the message encompasses the following steps –
  - **Step 1** – Calculation of the number of redundant bits.
  - **Step 2** – Positioning the redundant bits.
  - **Step 3** – Calculating the values of each redundant bit.
- Once the redundant bits are embedded within the message, this is sent to the user.

Hamming Code

Step 1 – Calculation of the number of redundant bits.

To correct an error, the receiver reverses the value of the altered bit. To do so, it must know which bit is in error.

Number of redundancy bits needed

- Let data bits = m
- Redundancy bits = r

$\therefore$ Total message sent = $m+r$

The value of r must satisfy the following relation:

$$2^r \geq m+r+1$$

Hamming Code-Finding no of Redundancy bits

$$2^r \geq m+r+1$$

- Suppose the number of data bits is 7, then the number of redundant bits can be calculated using: = 2^4 ≥ 7 + 4 + 1 Thus, the number of redundant bits= 4 **Parity bits.** A parity bit is a bit appended to a data of binary bits to ensure that the total number of 1's in the data is even or odd. Parity bits are used for error detection.

Hamming Code-
Step 2 – Positioning the redundant bits.

- The *r* redundant bits placed at bit positions of powers of 2, i.e. 1, 2, 4, 8, 16 etc.

- They are referred as r_1 (at position 1), r_2 (at position 2), r_3 (at position 4), r_4 (at position 8) and so on.

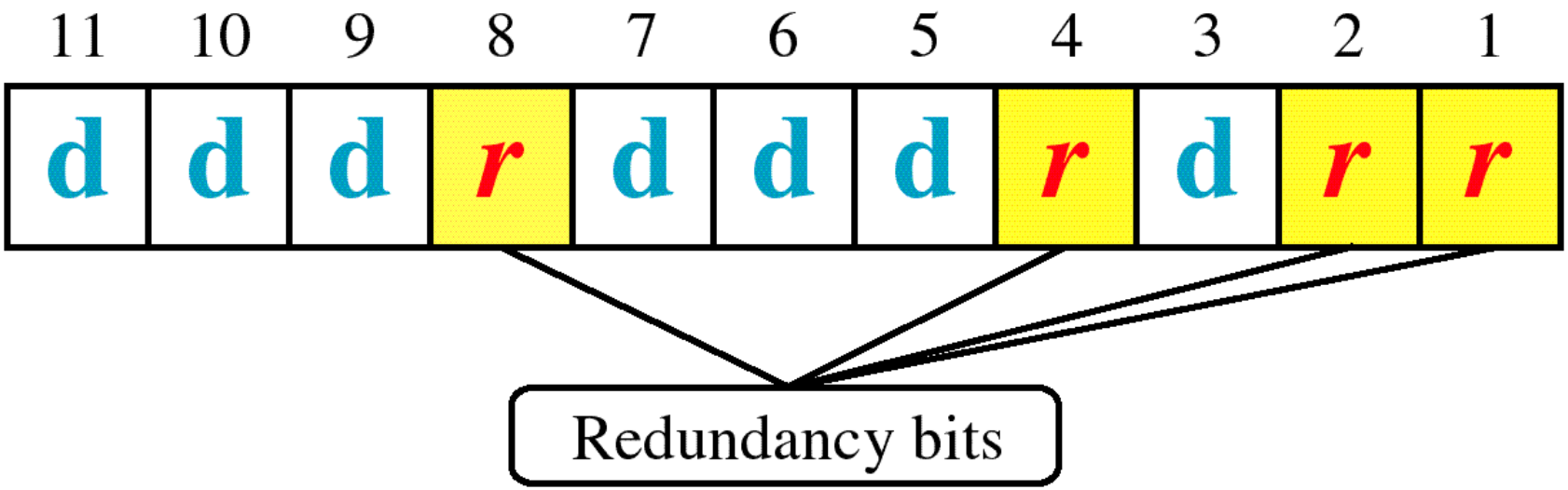
Hamming Code-
Step 3 – Calculating the values of each redundant bit.

- The redundant bits are parity bits. A parity bit is an extra bit that makes the number of 1s either even or odd. The two types of parity are –
- Even Parity – Here the total number of bits in the message is made even.
- Odd Parity – Here the total number of bits in the message is made odd.

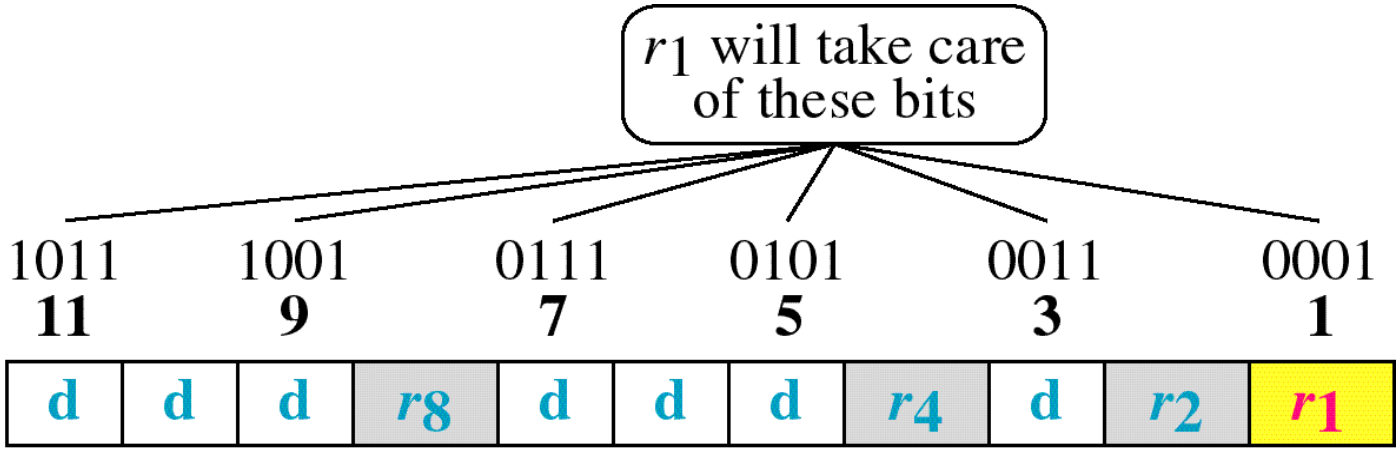
Step 3 – contd.

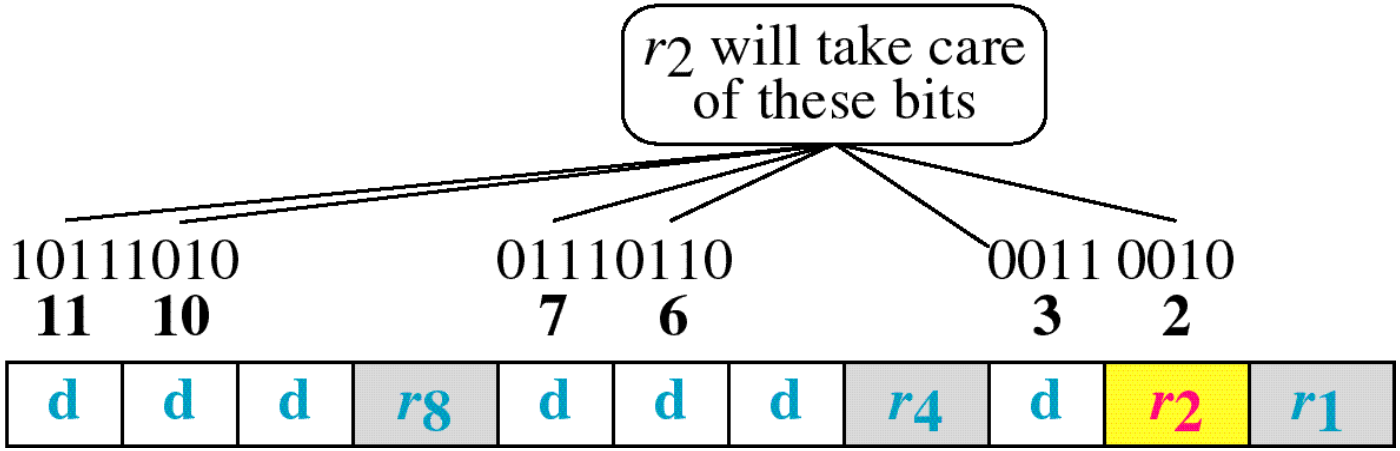
- Each redundant bit, ri, is calculated as the parity, generally even parity, based upon its bit position.
- It covers all bit positions whose binary representation includes a **1** in the ith position except the position of ri. Thus –
  - r1 is the parity bit for all data bits in positions whose binary representation includes a 1 in the least significant position excluding 1 (3, 5, 7, 9, 11 and so on)
  - r2 is the parity bit for all data bits in positions whose binary representation includes a 1 in the position 2 from right except 2 (3, 6, 7, 10, 11 and so on)
  - r3 is the parity bit for all data bits in positions whose binary representation includes a 1 in the position 3 from right except 4 (5-7, 12-15, 20-23 and so on)

Hamming Code

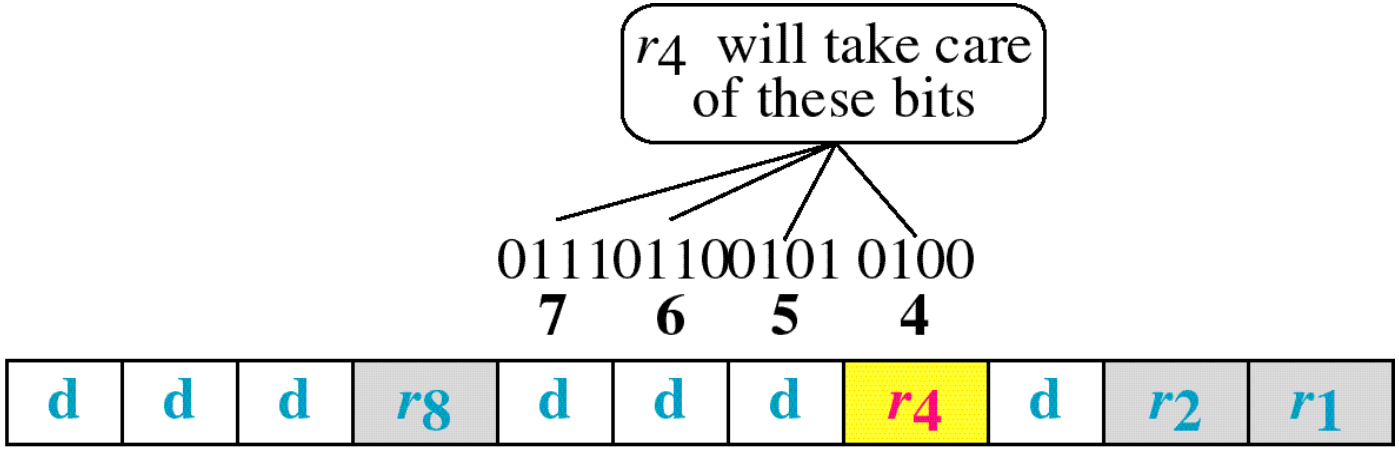


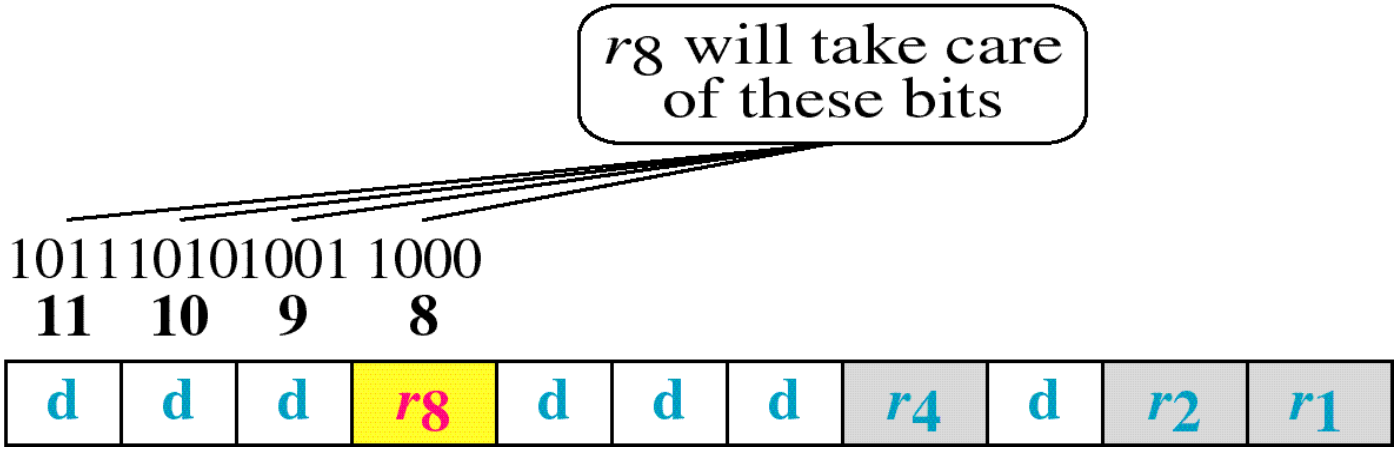
Hamming Code





Hamming Code





Hamming Code

- Write the bit positions starting from 1 in binary form (1, 10, 11, 100, etc).
- All the bit positions that are a power of 2 are marked as parity bits (1, 2, 4, 8, etc).
- All the other bit positions are marked as data bits.
- Each data bit is included in a unique set of parity bits, as determined its bit position in binary form.

Hamming Code

- **a.** Parity bit 1 covers all the bits positions whose binary representation includes a 1 in the least significant
position (1, 3, 5, 7, 9, 11, etc).
b. Parity bit 2 covers all the bits positions whose binary representation includes a 1 in the second position from
the least significant bit (2, 3, 6, 7, 10, 11, etc).
c. Parity bit 3 covers all the bits positions whose binary representation includes a 1 in the third position from the least significant bit (4–7, 12–15, 20–23, etc).

Hamming Code

- **d.** Parity bit 8 covers all the bits positions whose binary representation includes a 1 in the fourth position from
the least significant bit bits (8–15, 24–31, 40–47, etc).

Hamming Code : Decoding a message in Hamming Code

- Once the receiver gets an incoming message, it performs recalculations to detect errors and correct them. The steps for recalculation are −
  - Step 1 − Calculation of the number of redundant bits.
  - Step 2 − Positioning the redundant bits.
  - Step 3 − Parity checking.
  - Step 4 − Error detection and correction

Hamming Code : Decoding a message in Hamming Code

- **<u>Step 1 – Calculation of the number of redundant bits</u>**
- Using the same formula as in encoding, the number of redundant bits are ascertained.

- $2r \geq m + r + 1$ where m is the number of data bits and r is the number of redundant bits.

- **<u>Step 2 – Positioning the redundant bits</u>**
- The r redundant bits placed at bit positions of powers of 2, i.e. 1, 2, 4, 8, 16 etc.

Hamming Code : Decoding a message in Hamming Code

- **<u>Step 3 – Parity checking</u>**
- Parity bits are calculated based upon the data bits and the redundant bits using the same rule as during generation of c1,c2 ,c3 ,c4 etc. Thus

- c1 = parity(1, 3, 5, 7, 9, 11 and so on)

- c2 = parity(2, 3, 6, 7, 10, 11 and so on)

- c3 = parity(4-7, 12-15, 20-23 and so on)

Hamming Code : Decoding a message in Hamming Code

- **<u>Step 4 – Error detection and correction</u>**
- The decimal equivalent of the parity bits binary values is calculated.
- If it is 0, there is no error.
- Otherwise, the decimal value gives the bit position which has error. The bit is flipped to get the correct message.

9.b) Suppose you want to transmit the message 1011 0010 0100 1011 and protect it from errors using the CRC8 polynomial X^8 +X^2 +X^1 +1. CO3 BL3

i. Use polynomial long division to determine the message that should be transmitted. [5]

ii. Suppose the leftmost bit of the message is inverted due to noise on the transmission link. What is the result of the receiver's CRC calculation? How does the receiver know that an error has occurred? [5]

Given the data word 1111011111 and the divisor 11011,

i. Show the generation of the code word at the sender site (using binary division). (4 Marks)

ii. Show the checking of the code word at the receiver site (assume no error). (3 Marks)